Программная обработка CSV, конвейер коллекций и надежность обмена

Экспорт и потоковое чтение таблиц CSV, трансформация списков словарей и тестирование целостности файлов

Практикуемые темы и применяемые инструменты

ТЕМА 1	**Программная обработка данных в формате CSV:** подключение встроенного модуля `csv`, запись двумерных таблиц через `csv.writer` с разделителем `delimiter=";"` и параметром `newline=""`, потоковое чтение через `csv.reader` и отделение шапки функцией `next()`.
ТЕМА 2	**Обобщающий практикум по коллекциям и файлам:** очистка строковых полей методами `.strip()` и `.upper()`, приведение типов к `int`, обогащение словарей расчетными ключами, сортировка через `.sort(key=lambda ...)` и группировка сумм через словарь-накопитель.
ТЕМА 3	**Тестирование корректности обмена данными:** перехват исключений `FileNotFoundError` и `ValueError` в блоках `try-except`, отсев поврежденных строк в карантин без остановки цикла, экспорт через `csv.DictWriter`, проверка `Round-Trip` оператором `assert` и расчет хэша `hashlib.sha256()`.

Описание работы

Необходимо разработать единую программу в файле `csv_pipeline_audit.py`, объединяющую все три изученные темы в сквозной алгоритм обработки складских поставок. Программа последовательно создает исходный CSV-файл с «сырыми» и поврежденными строками (Часть 1), пропускает его через защищенный фильтр с очисткой, сортировкой и группировкой в памяти (Часть 2), после чего экспортирует чистую базу и проверяет ее целостность (Часть 3).

Часть 1. Программная обработка данных в формате CSV

ШАГ 1.1	**Подготовка исходной матрицы данных:** подключите модули `csv` и `hashlib`. Создайте переменную `raw_deliveries`, хранящую вложенный список списков (шапку и 5 строк поставок, включая одну намеренно поврежденную запись): • Шапка таблицы: `["code", "title", "category", "qty", "price"]` • Запись 1: `[" sw-01 ", "Коммутатор; 24 порта", "Сеть", "4", "12500"]` • Запись 2: `["srv-02", "Сервер стоечный", "Серверы", "2", "95000"]` • Запись 3 (битая цена): `["cab-03", "Патч-корд 5м", "Сеть", "20", "ошибка_цены"]` • Запись 4: `["ups-04", "ИБП модульный; 3000VA", "Питание", "3", "32000"]` • Запись 5: `[" srv-05 ", "Дисковая полка", "Серверы", "1", "64000"]`
ШАГ 1.2	**Экспорт матрицы в CSV-файл:** откройте файл `"incoming_stock.csv"` для перезаписи в кодировке `utf-8` с обязательным аргументом `newline=""`. Создайте объект записи `csv.writer` с разделителем `delimiter=";"` и выгрузите в файл весь список `raw_deliveries` за один вызов пакетного метода.
ШАГ 1.3	**Потоковое чтение и извлечение заголовка:** откройте `"incoming_stock.csv"` для чтения, создайте поток `csv.reader` (с тем же разделителем) и с помощью функции `next()` извлеките верхнюю строку в переменную `header`. Остальные строки сохраните в список `loaded_rows`. Выведите в консоль список колонок `header` и количество прочитанных строк данных `len(loaded_rows)`.

Часть 2. Обобщающий практикум по коллекциям, очистке и фильтрации

Продолжайте писать код в файле `csv_pipeline_audit.py`. На этом этапе необходимо разобрать список строк `loaded_rows`, защитить цикл от нечисловой цены, очистить текстовые поля и выполнить аналитические расчеты в коллекциях.

ШАГ 2.1	**Валидация строк в цикле и нормализация данных:** 1. Создайте два пустых списка: `clean_items` (для корректных карточек-словарей) и `quarantine_log` (для отбракованных записей). 2. Переберите список `loaded_rows` в цикле. Внутри цикла разместите конструкцию `try-except ValueError`. 3. В блоке `try` преобразуйте количество (индекс 3) и цену (индекс 4) к целому типу `int`. Очистите код товара (индекс 0) от краевых пробелов и переведите его в верхний регистр (`.strip().upper()`), а название (индекс 1) и категорию (индекс 2) очистите методом `.strip()`. 4. Рассчитайте общую стоимость партии `total = qty * price` и добавьте в `clean_items` словарь со строго заданными ключами: `"code"`, `"title"`, `"category"`, `"qty"`, `"price"`, `"total"`. 5. В блоке `except ValueError` (если цена или количество не являются числом) добавьте проблемный код товара `row[0].strip()` в список `quarantine_log`. 6. После цикла выведите в консоль количество принятых карточек и список кодов, попавших в карантин.
ШАГ 2.2	**Сортировка списка словарей по ключу:** 1. Отсортируйте список `clean_items` на месте методом `.sort()` по убыванию ключа итоговой стоимости партии `"total"` (используйте анонимную функцию `lambda` и флаг обратного порядка). 2. Выведите в консоль данные лидера рейтинга (элемент с индексом `0`): его код, название и стоимость партии.
ШАГ 2.3	**Фильтрация крупных поставок:** 1. Создайте список `priority_codes` и в цикле соберите в него только коды (`"code"`) тех позиций из `clean_items`, у которых стоимость партии `"total"` строго больше `60000` руб. 2. Выведите полученный список `priority_codes` в консоль.
ШАГ 2.4	**Агрегация (группировка) расходов по категориям:** 1. Создайте пустой словарь-накопитель `category_totals`. 2. Переберите карточки в `clean_items` и для каждой категории (ключ `"category"`) накопите суммарную стоимость `"total"` (используйте безопасный метод `.get(cat, 0)` или проверку `if cat not in ...`). 3. Выведите получившийся словарь `category_totals` в консоль.

Переход к Части 3 (контроль файлового доступа, экспорт словарей и тестирование целостности):
После очистки и сортировки в оперативной памяти сформирован эталонный список словарей `clean_items`. В заключительной части необходимо протестировать защиту от отсутствующих файлов, выгрузить чистую коллекцию через `DictWriter` и программно доказать, что при сохранении и обратном чтении данные не исказились.

Часть 3. Тестирование корректности обмена и целостности данных

ШАГ 3.1	**Проверка перехвата ошибки отсутствующего файла:** напишите блок `try-except FileNotFoundError`, который пытается открыть заведомо несуществующий резервный файл `"backup_archive.csv"` в режиме чтения `"r"`. При перехвате исключения выведите в консоль сообщение: `"Проверка защиты: файл backup_archive.csv не найден, работа продолжена"`.
ШАГ 3.2	**Выгрузка чистой базы через DictWriter:** задайте список названий колонок `fields = ["code", "title", "category", "qty", "price", "total"]`. Откройте файл `"verified_stock.csv"` для записи, создайте объект `csv.DictWriter` (с разделителем `";"`), запишите строку заголовков методом `.writeheader()` и выгрузите все карточки из `clean_items` методом `.writerows()`.
ШАГ 3.3	**Сквозной тест Round-Trip и проверка через assert:** 1. Создайте пустой список `restored_items` и прочитайте в него файл `"verified_stock.csv"` с помощью `csv.DictReader` (с разделителем `";"`). 2. Для каждой прочитанной записи приведите числовые поля `"qty"`, `"price"` и `"total"` обратно к типу `int`, чтобы типы полностью совпадали с оригиналом. 3. Выполните автоматическую самопроверку с помощью инструкции `assert clean_items == restored_items`.
ШАГ 3.4	**Вычисление цифрового отпечатка SHA-256:** откройте файл `"verified_stock.csv"` в бинарном режиме чтения `"rb"`, вычислите хэш-сумму с помощью `hashlib.sha256(bf.read()).hexdigest()` и сохраните ее в переменную `file_hash`. Выведите в консоль подтверждение успешного теста и первые 16 символов хэша `file_hash[:16]`.

Ожидаемый результат выполнения программы в терминале

После запуска файла `csv_pipeline_audit.py` сверьте вывод вашей программы с эталонным протоколом терминала:

ВЫВОД В ТЕРМИНАЛ: РЕЗУЛЬТАТ РАБОТЫ CSV_PIPELINE_AUDIT.PY SUCCESS

```
Шапка таблицы: ['code', 'title', 'category', 'qty', 'price'] | Строк данных: 5
Принято позиций: 4 | В карантине (ошибка данных): ['cab-03']
Лидер по стоимости: SRV-02 (Сервер стоечный) -> 190000 руб.
Крупные поставки (>60000 руб.): ['SRV-02', 'UPS-04', 'SRV-05']
Сводка по категориям: {'Серверы': 254000, 'Питание': 96000, 'Сеть': 50000}
Проверка защиты: файл backup_archive.csv не найден, работа продолжена
Тест целостности Round-Trip: УСПЕШНО (4 из 4 записей идентичны)
Контрольная сумма SHA-256 (первые 16 симв.): [16 символов вашего хэша]
```